



“Organiza hoy, gana mañana.”

Celin Eduardo Angel Vera
Nelson Alberto Carranza Puente
Elsy Jazmin Hernandez Ignacio
Adriana Yaneth Martinez Santiago
Lucero Fernanda Rodriguez Rodriguez
Alberto Jaret Vazquez Tovar

Índice

Introducción.....	3
Descripción del problema o necesidad a solucionar.....	4
Lógica de negocio (flujo del proceso sin usar la app).....	5
Justificación.....	6
Objetivo.....	7
Diagramas de flujo de uso de la aplicación (como se automatiza y mejora el proceso con el uso de la tecnología).....	8
Descripción del desarrollo de la aplicación por ejemplo metodología, arquitectura, flujo de datos, estructura, esquema de la base de datos.....	9
❖ Metodología-Scrum.....	9
➢ Ceremonias aplicadas.....	9
➢ Artefactos Scrum utilizados.....	10
❖ Arquitectura de la Aplicación — MVVM.....	11
➢ Descripción del patrón MVVM.....	11
➢ Diagrama MVVM del proyecto.....	12
➢ Beneficios de usar MVVM en Flutter.....	13
❖ Integración con Supabase — Flujo de Datos y Realtime.....	13
➢ Servicios de Supabase utilizados.....	13
➢ Flujo de datos — Carga de tareas con Realtime.....	14
➢ Descripción del flujo paso a paso.....	14
❖ Esquema de la Base de Datos.....	15
➢ Política de seguridad — Row Level Security (RLS).....	15
➢ Diagrama del esquema.....	15
➢ Buenas prácticas aplicadas en la BD.....	16
❖ Estructura de Carpetas del Proyecto.....	17
➢ Diagrama de la estructura.....	17
➢ Descripción de directorios.....	18
➢ Principios de organización aplicados.....	18
❖ Patrones de Diseño Aplicados.....	19
Pruebas (descripción de pruebas con evidencias y mención de errores o aspectos a mejorar).....	20

Introducción

El control adecuado de inventarios y el registro de ventas son elementos fundamentales para el buen funcionamiento de cualquier negocio, especialmente en tiendas pequeñas donde la administración suele realizarse de manera manual. Este tipo de manejo aunque es muy común puede generar errores en el registro de productos, pérdida de información y desorganización, lo que afecta directamente la toma de decisiones y las ganancias del mismo negocio.

Ante esta problemática, se propone el desarrollo e implementación de un sistema digital de control de inventario y registro de ventas, el cual estará conformado por dos módulos principales: un módulo administrativo instalado en una computadora y un módulo móvil destinado al escaneo de códigos de barras mediante un teléfono celular. El módulo móvil permitirá capturar la información de los productos de manera rápida y automática, enviándola al sistema principal, donde se visualizarán los datos y se podrá realizar el cobro de forma organizada.

Por otra parte, el módulo administrativo permitirá gestionar el inventario en tiempo real, facilitando el registro, consulta, modificación y eliminación de productos. Además, integrará herramientas de análisis mediante reportes y gráficas que mostrarán información relevante, como los productos más vendidos, los de menor rotación, el stock disponible y los ingresos generados en distintos periodos.

Este sistema busca dar solución a los problemas identificados en una tienda específica donde actualmente todo el proceso se realiza de manera manual, provocando errores en el registro de ventas y en el surtido de productos. De esta manera, el proyecto está dirigido a propietarios y encargados de pequeños negocios que requieran una herramienta accesible, fácil de usar y eficiente, que les permita mejorar la organización, optimizar recursos y tomar decisiones más acertadas para el crecimiento de su negocio.

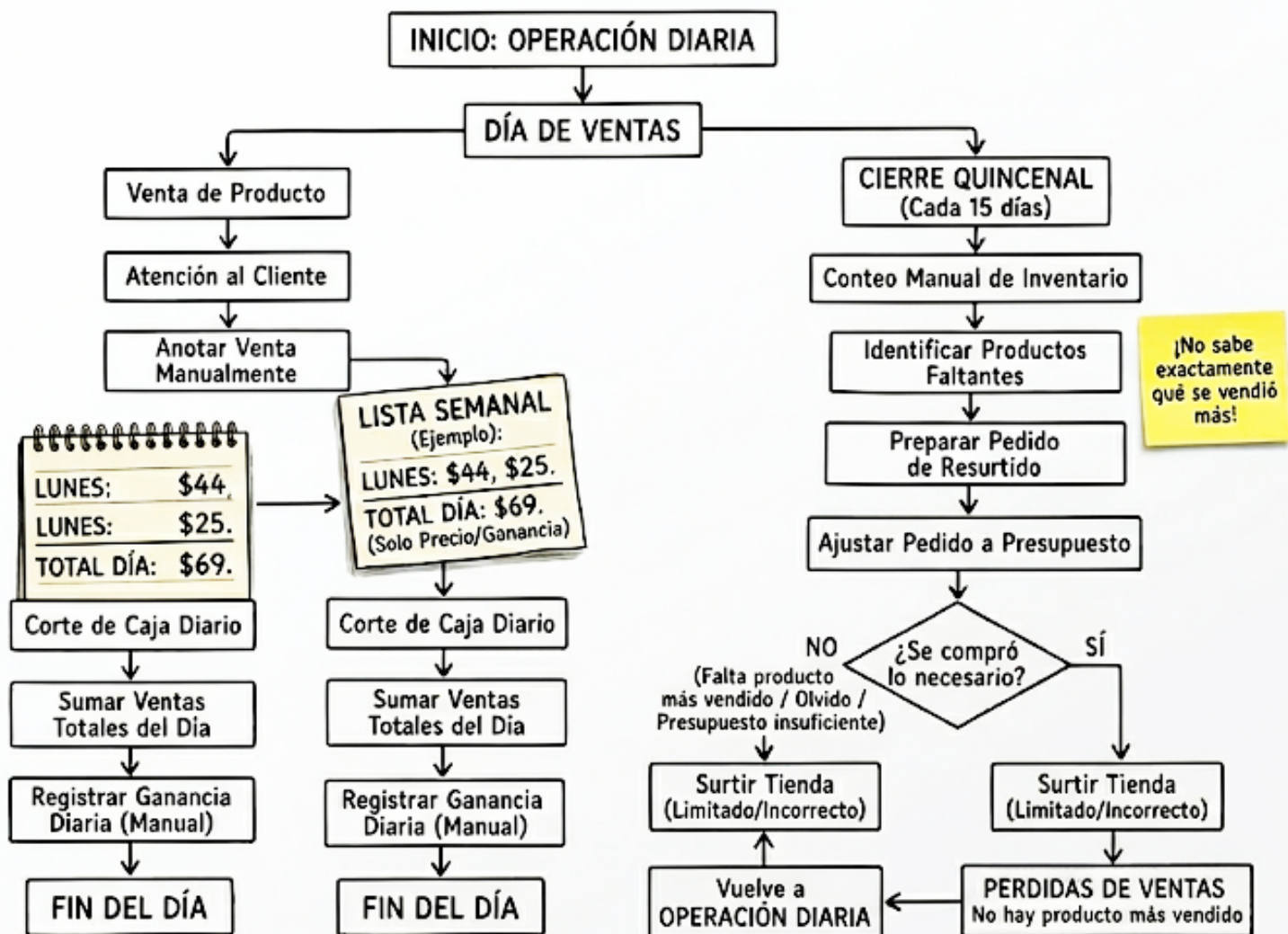
Descripción del problema o necesidad a solucionar

En muchas tiendas pequeñas, el registro de ventas y el control de inventario se realiza de forma manual, lo que provoca errores al momento de registrar las ventas, generando pérdida de información y desorganización en el manejo de los productos. Además, no se cuenta con un sistema que permita visualizar de manera clara los productos vendidos ni las ganancias generadas, lo que dificulta el proceso de surtido y puede afectar directamente los ingresos del negocio.

El problema se identifica en una tienda específica donde todo el proceso se lleva a cabo de forma manual. Las ventas y ganancias se registran día con día de manera escrita, lo que aumenta la probabilidad de errores. Asimismo, al momento de surtir productos, no se tiene un control preciso, lo que ocasiona la compra de productos innecesarios o la falta de aquellos que sí se requieren.

Por ello, el proyecto está dirigido a propietarios y encargados de tiendas pequeñas o negocios locales que necesiten una herramienta que les permita llevar un control fácil, rápido y organizado del inventario y las ventas. El sistema propuesto está diseñado para ser utilizado por personas sin experiencia técnica, con el fin de mejorar la administración del negocio, optimizar recursos y facilitar la toma de decisiones.

Lógica de negocio (flujo del proceso sin usar la app)



Justificación

El proyecto consiste en el desarrollo e implementación de un sistema digital de control de inventario y registro de ventas, conformado por un módulo administrativo instalado en una computadora y un módulo móvil complementario destinado al escaneo de códigos de barras mediante un teléfono celular.

El módulo móvil permitirá capturar los códigos de barras de los productos utilizando la cámara del dispositivo, enviando automáticamente la información al sistema principal. Una vez escaneados los productos, estos se visualizarán en la interfaz del sistema administrativo, donde se mostrará su información correspondiente y se habilitará la opción de realizar el cobro de manera rápida y organizada mediante un botón de confirmación de venta.

El módulo administrativo será el encargado de gestionar el inventario general del establecimiento, permitiendo registrar, consultar, modificar y eliminar productos, así como mantener actualizado el control de existencias en tiempo real conforme se realicen las ventas. Además, el sistema incluirá un apartado de reportes que permitirá visualizar estadísticas mediante gráficas sobre los productos más vendidos, los de menor rotación, la cantidad disponible en stock y el total de ingresos generados en determinados periodos.

La integración de ambos módulos permitirá mantener sincronizada la información del inventario y las ventas, facilitando el control de los productos escaneados, agilizando el proceso de cobro y proporcionando herramientas visuales de análisis que contribuyan a mejorar la administración y organización del negocio.

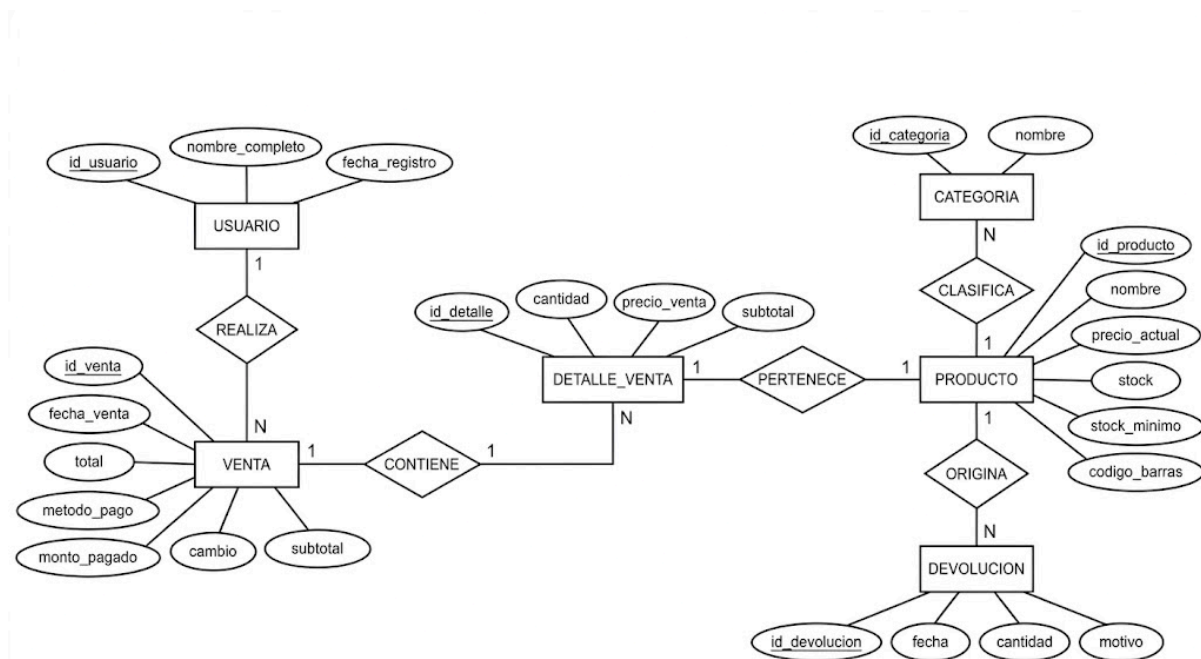
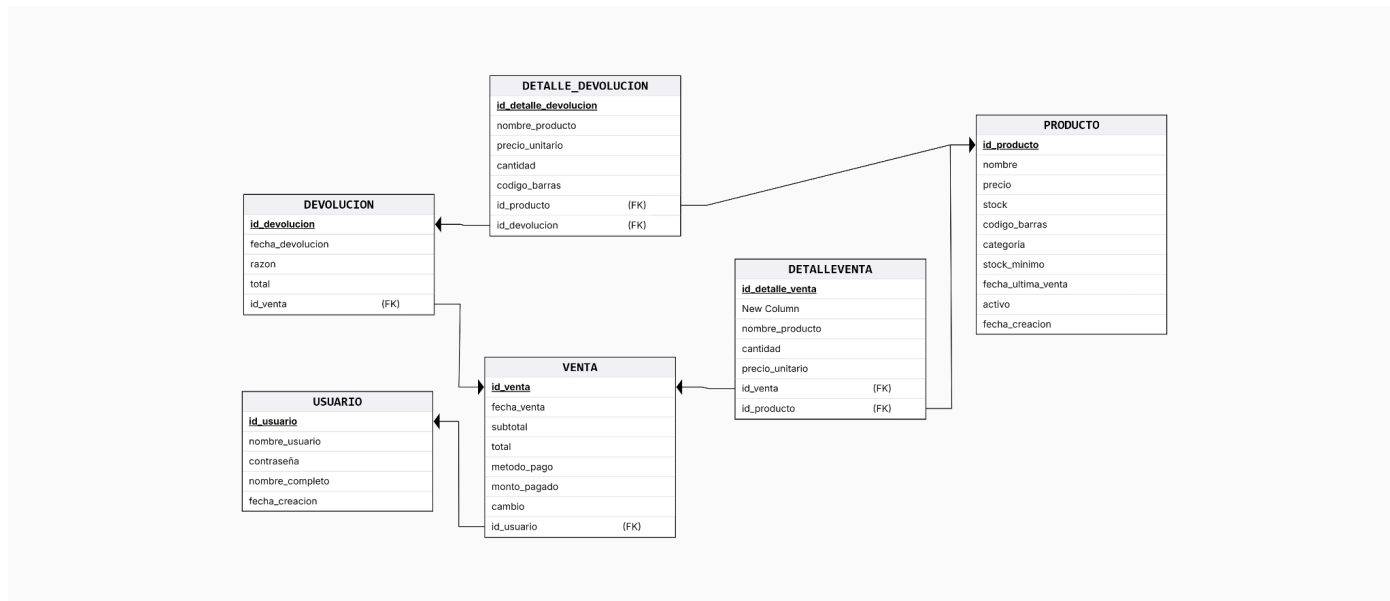
Objetivo

La aplicación facilita el trabajo del dueño de la tienda mediante la automatización del registro de ventas y el control de inventario, eliminando el conteo manual para reducir errores. De esta manera, se ahorra tiempo y se mantiene un control más preciso de los productos vendidos y sus precios.

Además, la información se presentará de forma clara y sencilla mediante gráficas, lo que permitirá comprender mejor el comportamiento de las ventas, identificar pérdidas o ganancias y tomar decisiones más acertadas para el crecimiento del negocio.

Diagramas de flujo de uso de la aplicación (como se automatiza y mejora el proceso con el uso de la tecnología)

1. Diagrama Conceptual (Modelo Entidad-Relación)



Descripción del desarrollo de la aplicación por ejemplo metodología, arquitectura, flujo de datos, estructura, esquema de la base de datos

❖ Metodología-Scrum

La Metodología Scrum fue la que se utilizó para este proyecto, ya que nos permitió poder llevar un sistema de control de registro de las actividades, en el cual nos brindó poder organizar los trabajos en ciclos cortos, facilitando la entrega de las actividades, así como la detección y corrección de errores para poder modificarlo a tiempo antes de su entrega.

Gracias a Scrum, el equipo pudo adaptarse a tener que priorizar las funciones más importantes del sistema y poder así mejorar continuamente el proyecto conforme avanzaban las iteraciones.

➤ ¿Qué es Scrum?

Scrum es un marco de trabajo (framework) ágil que se usa para gestionar proyectos, especialmente en el desarrollo de software.

➤ Ceremonias aplicadas

Ceremonia	Frecuencia	Descripción
Sprint Planning	Inicio de cada Sprint	Se definieron las funcionalidades a desarrollar en cada etapa del proyecto, como el registro de productos, ventas y reportes, así como el tiempo estimado para cada tarea.
Daily (Reunión diaria)	Diario (breve)	Se dio seguimiento al avance del sistema, comentando lo realizado, lo pendiente y los problemas encontrados durante el desarrollo.
Sprint Review	Fin de cada Sprint	Se revisaron las funcionalidades implementadas, verificando que el sistema funcionara correctamente, como el escaneo de productos y la actualización del inventario.
Sprint Retrospective	Fin de cada Sprint	Se analizaron los errores y aciertos del equipo para mejorar el proceso en los siguientes Sprints.
Backlog del Producto	Durante todo el proyecto	Se organizó y priorizó la lista de funcionalidades del sistema, dando mayor importancia a las necesidades faltantes del negocio.

➤ Artefactos Scrum utilizados

• Product Backlog

Se creó una lista priorizada de todas las funcionalidades del sistema, como el registro de nuevos productos, control de inventario, escáner de códigos de barras, y generación de reportes.

• Sprint Backlog

En cada Sprint se seleccionaron las tareas más importantes del Product Backlog, organizándose según su prioridad. Por ejemplo, primero se desarrollaron funciones básicas como el registro de productos y posteriormente el escaneo y los reportes.

• Incremento

Al finalizar cada Sprint se obtuvo una parte funcional del sistema, como el módulo de inventario o el registro de ventas, permitiendo que el sistema fuera creciendo hasta estar completo.

• Definition of Done (DoD)

Para considerar una tarea como terminada, se ocupó: que la funcionalidades funcionen correctamente, que no presente errores, y que cumpla con los requisitos establecidos.

➤ Roles del Equipo

Roles definidos que el Scrum aplicó en este proyecto, junto con las responsabilidades de cada miembro del equipo.

Rol	Nombre/Asignado	Responsabilidades Principales
Product Owner	Elsy Hernandez	Define y prioriza el Product Backlog. Representa las necesidades del cliente y valida las entregas de cada Sprint.
Scrum Master	Adriana Martinez	Facilita los Scrum, coordina al equipo y ayuda a eliminar impedimentos durante el desarrollo del proyecto.
Developer (Flutter)	Celin Angel	Diseña e implementa las pantallas, navegación y funcionalidades principales de la aplicación móvil.
Developer (Backend/Supabase)	Alberto Vazquez	Configura y administra la base de datos, autenticación y conexión del sistema mediante Supabase.
QA / Tester	Nelson Carranza	Realiza pruebas funcionales del sistema, detecta errores y valida el cumplimiento de la Definition of Done.
UI/UX Designer	Lucero Rodriguez	Diseña la interfaz visual, wireframes y prototipos para mejorar la experiencia del usuario en la aplicación.

❖ Arquitectura de la Aplicación — MVVM

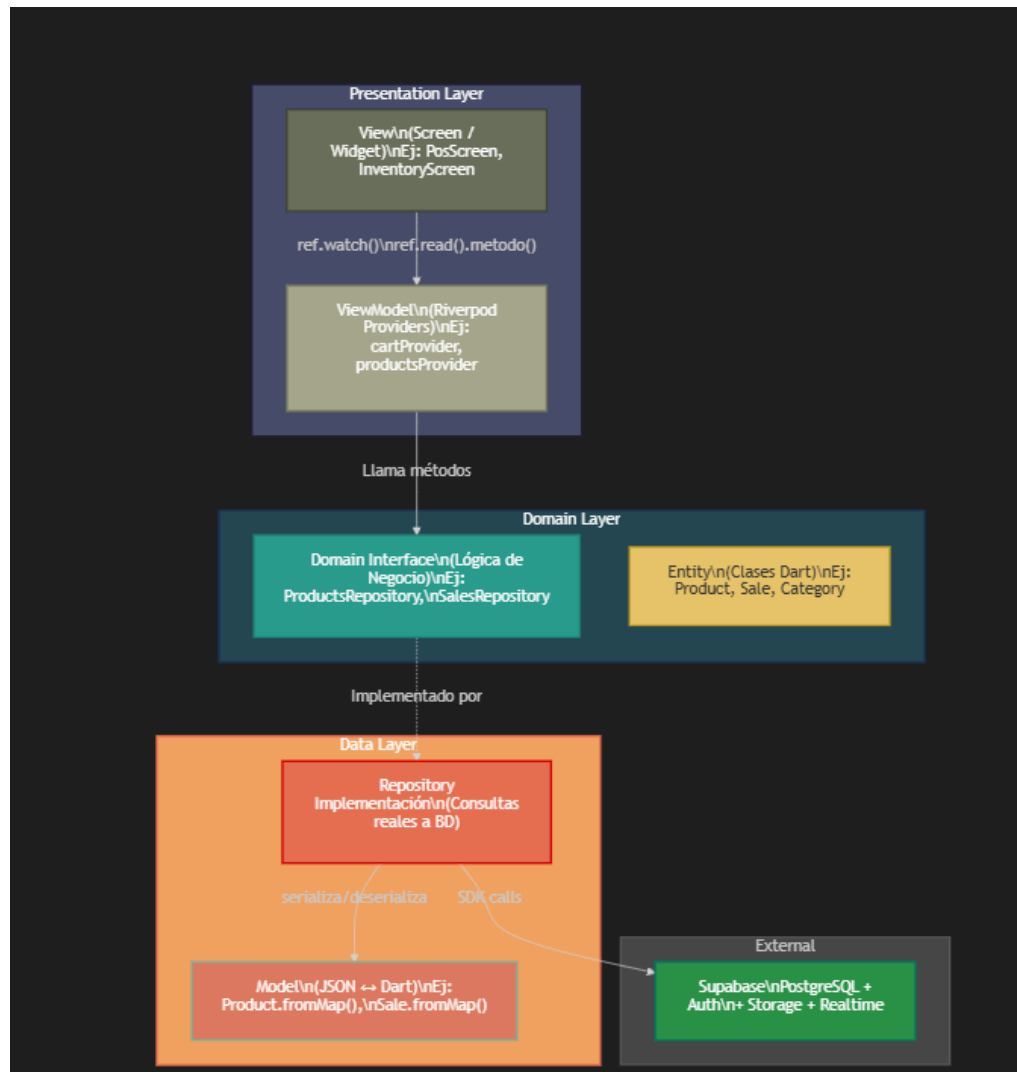
La aplicación fue desarrollada en MVVM (Model-View-ViewModel), ya que permite separar la lógica de negocio, la interfaz gráfica y el manejo de datos de manera organizada. Esto facilita el mantenimiento del sistema, mejora la comunicación entre el módulo administrativo y el módulo móvil, y permite realizar futuras actualizaciones de forma más sencilla y eficiente.

➤ Descripción del patrón MVVM

Capa	Responsabilidad	Descripción y tecnología usada
Model	Datos y lógica de negocio	Administra la información de productos, ventas e inventario, además del control de stock y registro de ventas mediante una base de datos segura.
View	Interfaz de usuario	Muestra las pantallas del sistema para registrar ventas, visualizar inventario, escanear códigos de barras y consultar reportes.
ViewModel	Mediador entre Model y View	Procesa las acciones del usuario, actualiza el inventario y muestra los cambios en tiempo real en la interfaz.
Repository	Abstracción del origen de datos	Actúa como intermediario entre la aplicación y la base de datos, gestionando la información de inventario y ventas.

➤ Diagrama MVVM del proyecto

El siguiente diagrama ilustra la relación entre las capas de la arquitectura y el flujo de comunicación con Supabase como backend.



➤ Beneficios de usar MVVM en Flutter

En el sistema de control de inventario y ventas, la arquitectura MVVM permitió separar la interfaz móvil, la lógica del escaneo y el manejo de datos de Supabase, facilitando un desarrollo más organizado.

Además:

- Ayudó a mantener el código ordenado y fácil de actualizar.
- Facilitó la conexión entre el módulo móvil y la base de datos.
- Permite manejar de forma más sencilla las ventas, productos y devoluciones.
- Hizo más fácil corregir errores y agregar nuevas funciones al sistema.

❖ Integración con Supabase — Flujo de Datos y Realtime

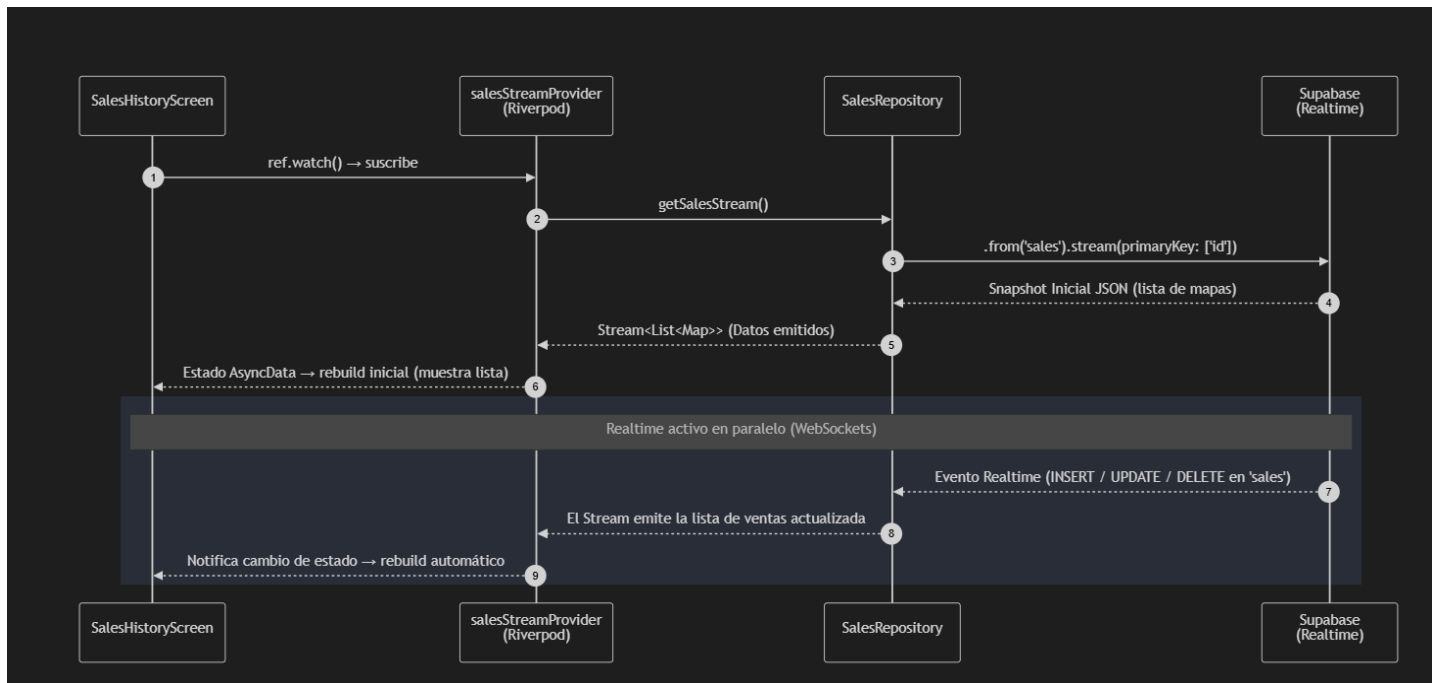
Supabase permite que toda la información relacionada con productos, ventas y usuarios se mantenga sincronizada entre el módulo administrativo (computadora) y el módulo móvil (escaneo de códigos de barras), asegurando que los datos estén siempre actualizados.

➤ Servicios de Supabase utilizados

Servicio	Uso en el proyecto
Authentication	Permite gestionar usuarios como administradores y encargados mediante correo y contraseña, brindando sesiones seguras y controlando el acceso al sistema de inventario y ventas.
PostgreSQL (DB)	Es la base de datos principal donde se guarda toda la información del sistema, como productos, precios, stock y ventas, con medidas de seguridad que controlan el acceso de cada usuario.
Realtime	Permite que el inventario y las ventas se actualicen en tiempo real, reflejando automáticamente los cambios en el stock cuando se escanean productos o se realizan ventas, sin necesidad de recargar la aplicación.
Storage	Se utiliza para guardar imágenes de productos, facilitando su identificación y mejorando la organización y búsqueda dentro del inventario.
Edge Functions	Se usan para ejecutar funciones del sistema en el servidor de Supabase, como calcular ventas, validar stock y generar reportes, sin depender del dispositivo del usuario.

➤ Flujo de datos — Carga de tareas con Realtime

El siguiente diagrama muestra cómo la aplicación carga los datos iniciales desde Supabase y se mantiene sincronizada en tiempo real con cualquier cambio en la base de datos.



➤ Descripción del flujo paso a paso

- **Suscripción (ref.watch):** Cuando el usuario entra a la pantalla SalesHistoryScreen o ReportsScreen, la vista se suscribe al proveedor salesStreamProvider usando ref.watch().
- **Llamada al Repositorio:** El proveedor solicita el flujo de datos llamando a getSalesStream() dentro del SalesRepository.
- **Suscripción a Supabase:** El repositorio usa la función nativa de Supabase .stream(primaryKey: ['id']) sobre la tabla sales. Esto hace dos cosas: trae los datos actuales y abre una conexión WebSocket.
- **Carga Inicial:** Supabase devuelve el estado actual de las ventas. La pantalla se construye (rebuild) y muestra los datos.
- **Realtime en paralelo:** La conexión se queda abierta en segundo plano. Si otro dispositivo registra una venta (un INSERT), Supabase detecta el cambio en la tabla sales y empuja el evento hacia la app.
- **Actualización Automática:** El Stream emite una nueva lista, Riverpod nota que los datos cambiaron, y automáticamente le dice a la pantalla que se vuelva a dibujar (rebuild automático) sin que el usuario tenga que recargar la página manualmente.

❖ Esquema de la Base de Datos

La base de datos del sistema está alojada en Supabase (PostgreSQL). Su estructura permite almacenar y administrar la información relacionada con los usuarios, productos, y registros del sistema. El diseño se realizó siguiendo principios de organización relacional para mantener la integridad y seguridad de los datos.

➤ Política de seguridad — Row Level Security (RLS)

Para garantizar la seguridad de la información, se implementó Row Level Security (RLS) en las tablas principales de la base de datos.

Las políticas de seguridad permiten que:

- Cada usuario autenticado únicamente puede acceder a la información relacionada con su cuenta.
- Las operaciones de lectura, inserción, actualización y eliminación están restringidas mediante el uso de:

```
user_id = auth.uid()
```

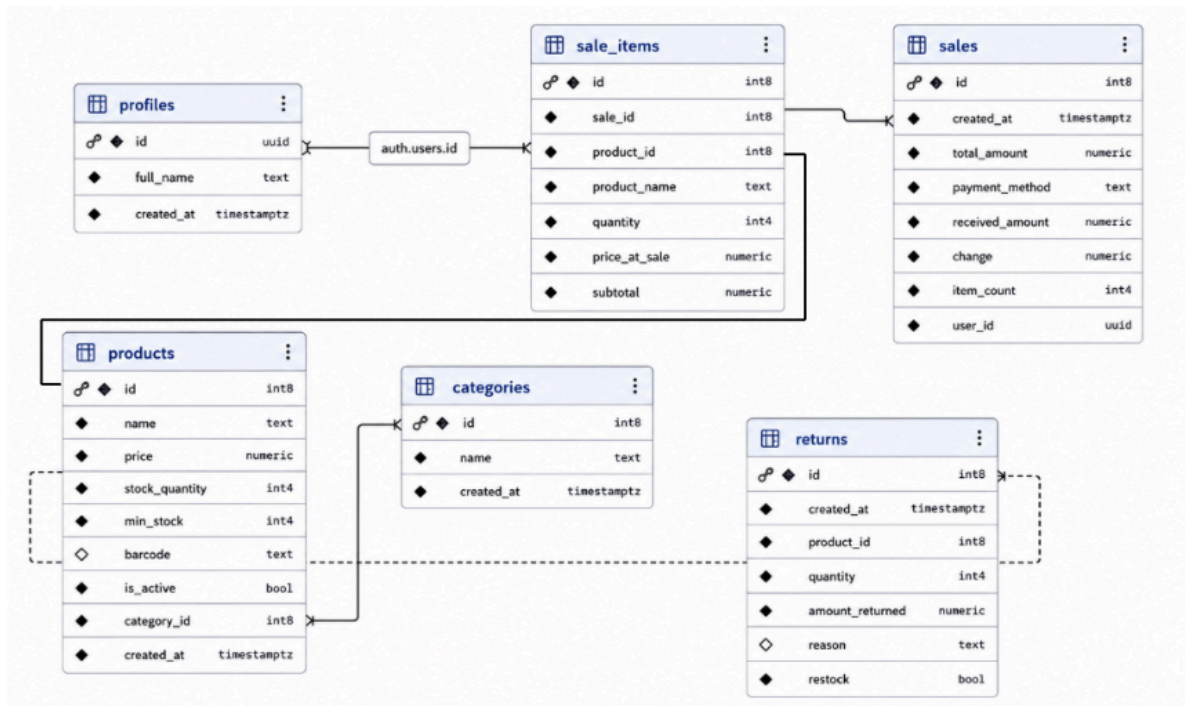
Esto asegura que los datos de ventas y movimientos del inventario pertenezcan únicamente al usuario del sistema.

Además:

- Los perfiles de usuario están vinculados directamente con `auth.users.id`.
- Las operaciones administrativas y procesos avanzados pueden ejecutarse mediante funciones seguras utilizando el rol `service_role`.

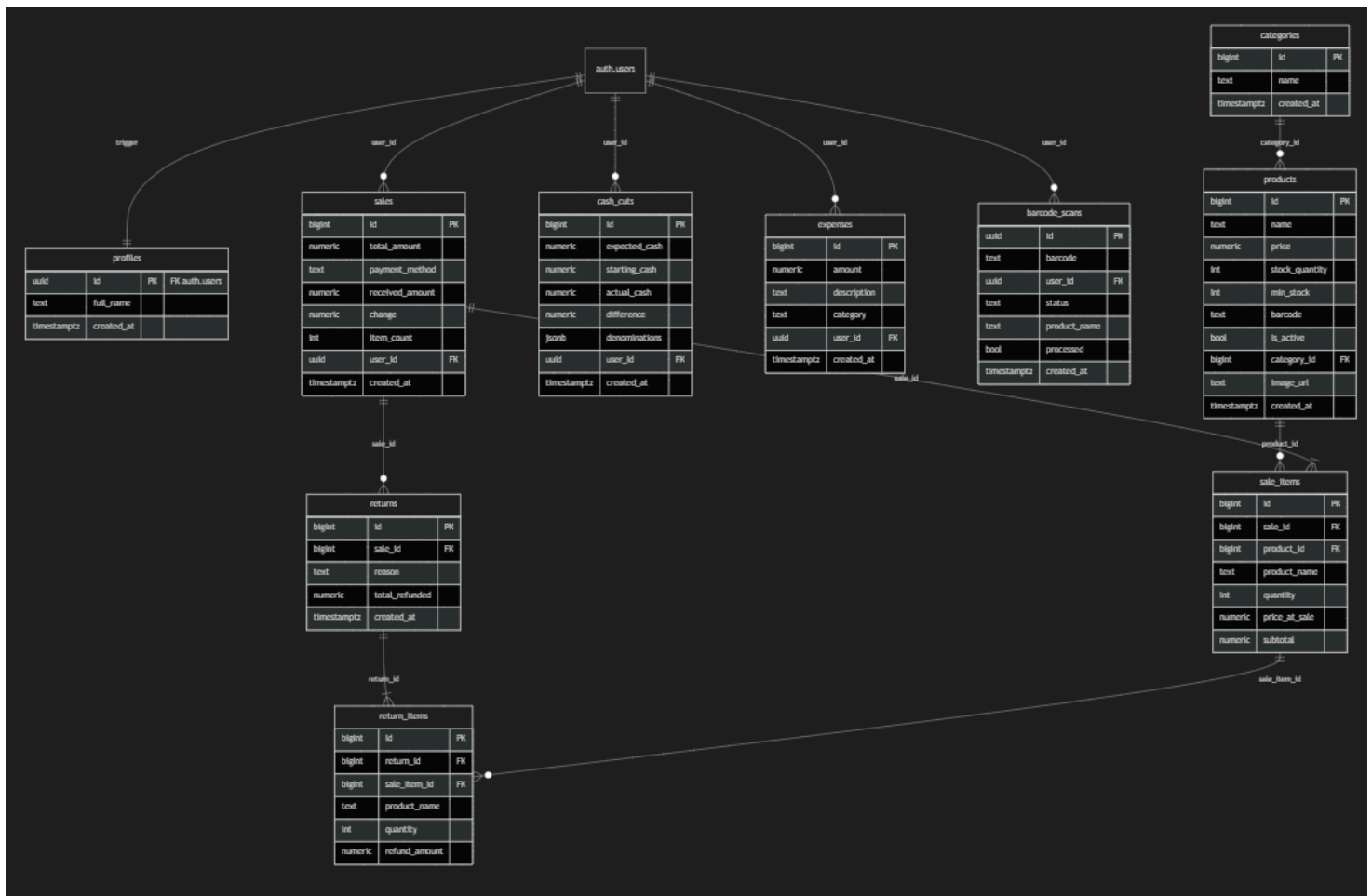
➤ Diagrama del esquema

El siguiente diagrama muestra las tablas principales de la base de datos y sus relaciones.



➤ Buenas prácticas aplicadas en la BD

- Uso de BigInt y UUID como claves primarias para escalabilidad y facilidad de relación con el sistema de autenticación de Supabase (Auth).
- Timestamps automáticos: La columna created_at se gestiona por defecto mediante funciones de base de datos (now) garantizando precisión en reportes y cortes de caja.
- Claves foráneas con ON DELETE CASCADE: Para mantener la integridad referencial. Por ejemplo, al eliminar una venta, se eliminan sus sale_items; lo mismo ocurre con los return_items de una devolución (returns).
- Campos JSONB (Datos semi-estructurados): Utilizados en cash_cuts (denominations) para guardar dinámicamente el conteo de billetes y monedas sin crear tablas intermedias innecesarias.
- Realtime y Sincronización: Tablas como barcode_scans y sales están preparadas para emitir eventos WebSocket, permitiendo que la interfaz de usuario se actualice al instante sin recargar la página.



❖ Estructura de Carpetas del Proyecto

La organización del código sigue una estructura modular inspirada en la arquitectura MVVM y en las buenas prácticas de desarrollo en Flutter. Cada carpeta tiene una responsabilidad específica, permitiendo mantener el proyecto ordenado, escalable y fácil de mantener.

➤ Diagrama de la estructura

```
lib/  
├─ dialogs/      ← add_product_dialog.dart, checkout_dialog.dart,  
held_carts_dialog.dart, ...  
├─ models/       ← product.dart, sale.dart, cart_item.dart, category.dart, ...  
├─ providers/    ← cart_notifier.dart, products_provider.dart,  
theme_notifier.dart, ...  
├─ repositories/ ← auth_repository.dart, products_repository.dart,  
sales_repository.dart  
├─ screens/      ← pos_screen.dart, inventory_screen.dart, login_screen.dart,  
reports_screen.dart, ...  
├─ theme/        ← app_theme.dart  
├─ utils/        ← receipt_generator.dart  
├─ widgets/      ← app_shell.dart, auth_gate.dart, animated_pressable.dart,  
numeric_keypad.dart, ...  
└─ main.dart
```

➤ Descripción de directorios

Carpeta / Archivo	Responsabilidad
lib/main.dart	Punto de entrada de la aplicación. Inicializa la conexión con Supabase, envuelve la app en <code>ProviderScope</code> (Riverpod) y lanza el widget raíz.
lib/dialogs/	Contiene cuadros de diálogo y ventanas modales personalizadas para interacciones específicas (ej. confirmación de cobro, ingreso de stock).
lib/models/	Clases Dart que representan las entidades del dominio (ej. <code>Product</code> , <code>Sale</code>). Incluyen los métodos <code>fromMap</code> y <code>toMap</code> para la conversión de datos (JSON) desde/hacia Supabase.
lib/providers/	Gestión del estado global y ViewModels utilizando Riverpod. Orquestan la lógica de negocio, consumen los Repositories y exponen los datos (o Streams) a la UI.
lib/repositories/	Implementaciones concretas de acceso a datos. Contienen las llamadas directas al SDK de Supabase (<code>select</code> , <code>insert</code> , <code>update</code> , <code>delete</code> , <code>stream</code>).
lib/screens/	Pantallas principales de la aplicación. Cada archivo (View) corresponde a una ruta o funcionalidad principal (Punto de Venta, Inventario, Reportes).
lib/theme/	Elementos transversales de diseño de la interfaz: colores constantes, bordes y estilos globales definidos en <code>AppTheme</code> .
lib/utils/	Funciones auxiliares y servicios independientes de la interfaz de usuario, como la generación de recibos en PDF (<code>receipt_generator.dart</code>).
lib/widgets/	Componentes visuales (Widgets) reutilizables compartidos entre múltiples pantallas (botones animados, barras de navegación laterales, teclado numérico).

test/	Directorio destinado a las pruebas unitarias y de integración de la aplicación.
-------	---

➤ Principios de organización aplicados

- Organización Layer-first (Por Capas): Se optó por una estructura arquitectónica agrupada por capas técnicas (models/, screens/, providers/, repositories/). Esta aproximación es ideal para sistemas de tamaño mediano como un Punto de Venta (POS), ya que facilita la navegación rápida por tipo de responsabilidad.
- Principio de Responsabilidad Única (SRP): Cada archivo tiene un propósito único y claramente definido. Por ejemplo, un archivo en screens/ (como pos_screen.dart) se encarga exclusivamente de construir la interfaz gráfica, delegando el manejo del carrito a cart_notifier.dart y las consultas a la base de datos a sales_repository.dart.
- Inyección de Dependencias (mediante Riverpod): Las capas superiores (las vistas) no instancian directamente las clases de lógica concreta. En su lugar, consumen "Providers" (ej. ref.read(productsRepositoryProvider)). Esto reduce el acoplamiento y permite que las dependencias sean inyectadas o simuladas fácilmente.
- Separación de Concerns (UI Reactiva vs Lógica): Se utiliza un enfoque reactivo declarativo. La interfaz visual simplemente "escucha" los cambios (ref.watch), mientras que la manipulación de los datos y el estado de la aplicación reside en la capa de Providers (ViewModels), asegurando que la interfaz se actualice automáticamente sin intervención manual.

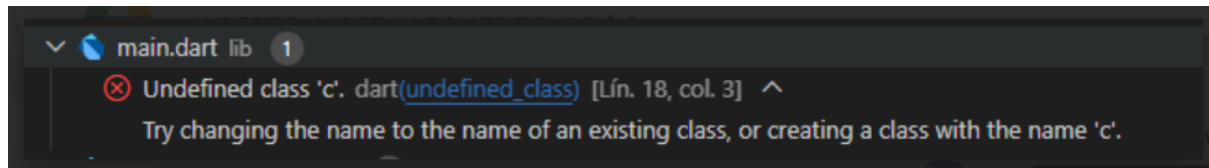
❖ Patrones de Diseño Aplicados

Además de la arquitectura MVVM, el proyecto utiliza distintos patrones de diseño para mejorar la organización, mantenimiento y funcionamiento del sistema.

Patrón	Categoría	Aplicación en el proyecto
Repository Pattern	Estructural	Permite manejar la conexión con Supabase sin afectar la interfaz o los ViewModels.
Observer (ChangeNotifier)	Comportamiento	Actualiza automáticamente la interfaz cuando cambia la información del inventario o ventas.
Singleton	Creacional	La conexión principal con Supabase se crea una sola vez y se comparte en toda la aplicación.
Dependency Injection	Estructural	Facilita enviar servicios y repositorios a los ViewModels mediante Provider o Riverpod.
Factory (fromJson/toJson)	Creacional	Los modelos convierten datos JSON de Supabase de forma organizada y segura.

Pruebas (descripción de pruebas con evidencias y mención de errores o aspectos a mejorar)

A continuación se detallan las pruebas más relevantes, los errores detectados y los aspectos identificados para futuras mejoras.



1. Prueba de Sincronización en Tiempo Real (Escáner a Caja)

Descripción de la prueba: Se simuló el escenario principal de la tienda: escanear productos rápidamente con el módulo móvil y verificar que se reflejaran instantáneamente en el carrito de compras del módulo administrativo usando Supabase Realtime.

Error detectado: Si el usuario escanea el código de barras de un producto de forma muy rápida y repetitiva (doble escaneo accidental), el sistema insertaba el producto dos veces en la base de datos de barcode_scans antes de que la UI pudiera bloquear la cámara, provocando cobros duplicados en el carrito.

Solución implementada: Se añadió una validación en la lógica de Riverpod (implementando un patrón Debounce) que ignora múltiples lecturas del mismo código dentro de una ventana de 500 milisegundos. Además, se añadió una columna booleana processed en la base de datos para garantizar que cada evento Realtime se cobre una sola vez.

2. Prueba de Integridad de Inventario (Manejo de Stock)

Descripción de la prueba: Se procedió a realizar una venta que exceda la cantidad de unidades disponibles en el inventario para evaluar la respuesta de la base de datos.

Error detectado: La versión inicial del repositorio (SalesRepository) descontaba el stock ciegamente. Si había 2 productos en stock y se vendían 3, el inventario quedaba en números negativos (stock_quantity = -1).

Solución implementada: Se modificó la función de actualización en Dart para usar la lógica matemática $\max(0, \text{currentStock} - \text{item.quantity})$. Esto previene que los números de stock bajen de cero en la base de datos y protege la integridad de la información.

Aspecto a mejorar: Como trabajo futuro, se planea implementar esta restricción no solo en el código de Flutter, sino directamente en Supabase usando restricciones CHECK o funciones RPC (Remote Procedure Calls) para manejar la concurrencia en caso de que dos cajeros vendan el mismo producto al mismo tiempo.

3. Prueba de Conectividad (Aspecto a Mejorar)

Descripción: Se probó el sistema desconectando temporalmente el equipo de la red Wi-Fi simulando una caída del internet local de la tienda.

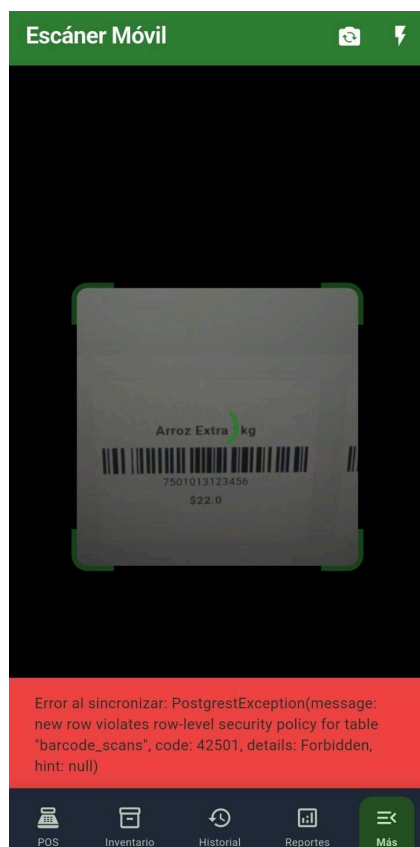
Error detectado: Al no tener internet, la aplicación no podía establecer conexión con Supabase. El intento de insertar una nueva venta (INSERT INTO sales) fallaba lanzando una excepción, y el cajero no podía terminar de cobrar al cliente.

Aspectos a mejorar (Deuda Técnica): Dado que la tienda física no puede detener sus operaciones por problemas de internet, el sistema debe ser capaz de operar "Offline". Se ha documentado como prioridad para la próxima iteración implementar almacenamiento local (usando SQLite o Hive) para guardar las ventas de forma temporal en el dispositivo y sincronizarlas automáticamente con Supabase una vez que se restablezca la conexión a internet.

4. Error al escanear

Se resolvió el error de sincronización del escáner mediante la configuración de políticas de Seguridad de Nivel de Fila (RLS) en la tabla barcode_scans de Supabase.

El problema radica en que las solicitudes de inserción desde la aplicación móvil eran rechazadas por falta de permisos explícitos; la solución consistió en aplicar sentencias SQL para habilitar políticas de tipo INSERT, SELECT y UPDATE para usuarios autenticados, asegurando que cada operario solo pueda gestionar sus propios registros mediante la validación del ID de usuario (auth.uid() = user_id), garantizando así la integridad y seguridad de los datos en tiempo real.



Interfaces de sistema y su funcionamiento